Training in Deployment: Human-in-the-loop Online Post-Training for Robot Manipulation

Chun Gu[1,2,3*], Xiaofei Wei[1,2,3*], Yiming Chen[2,3], Bozhou Zhang[1,3], Juntong Zhang[2,3], Xiatian Zhu[4], Li Zhang[1,2,3†]
[1]School of Data Science, Fudan University [2]Shanghai Innovation Institute [3]Logos Robotics Group
[4]University of Surrey

logosroboticsgroup.github.io/TIDE

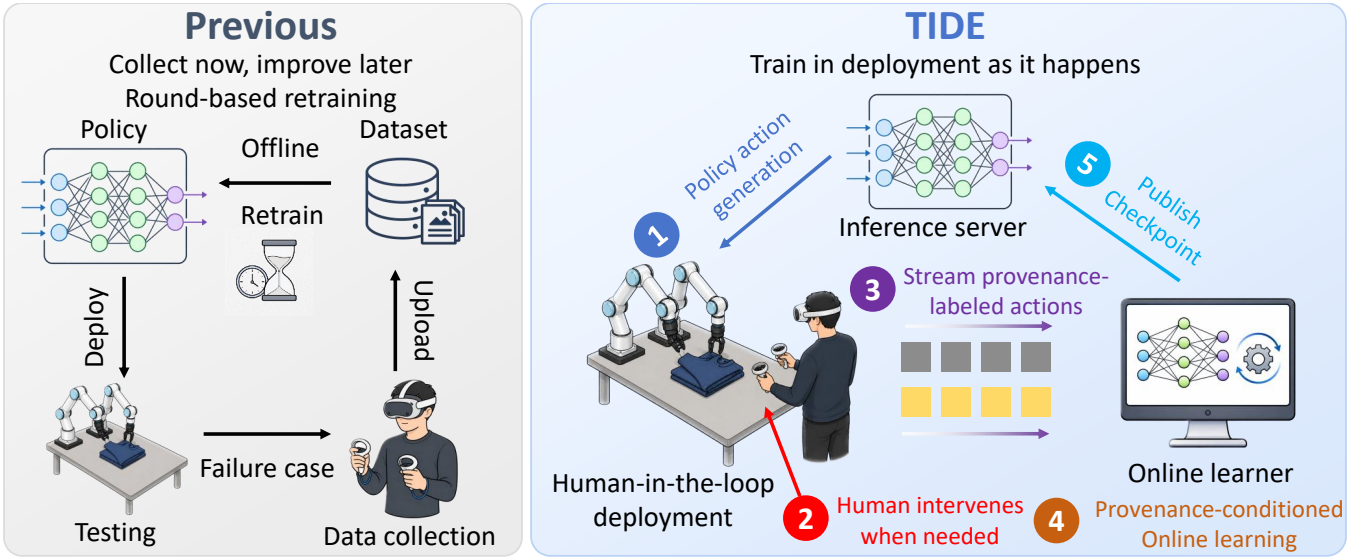


Fig. 1. **Training in Deployment.** Left: Round-based methods collect data for later offline retraining. Right: TIDE learns online from human interventions and policy executions, then redeploys updated checkpoints, making policy improvements available within the same deployment session.

***Abstract*— Large pretrained robot foundation models provide strong priors for manipulation, yet adapting them to target tasks with offline teleoperated demonstrations often fails to cover the contact-rich, recovery, and precision-critical states encountered during deployment. Existing methods mitigate this issue by augmenting offline demonstrations with human-in-the-loop data or online reinforcement learning. Despite this, offline post-training adapts slowly because it cannot immediately absorb deployment experience, while online RL typically relies on value functions or auxiliary actor-critic models, increasing both training complexity and instability. We present TIDE (Training In DEployment), a human-in-the-loop online post-training system. TIDE introduces a provenance-conditioned supervised learning framework that updates the policy online from human interventions and policy-generated rollouts collected during deployment, without a learned critic, reward model, or action scorer. Across four real-world bimanual manipulation tasks, TIDE improves milestone-normalized task progress and binary full-task completion over the warm-start policy and matched additional offline demonstrations, and performs comparably to the fixed-policy collection baseline while making updated checkpoints available during deployment. Each task-specific online post-training session, from the start of deployment rollouts until the final checkpoint is available, completes in less than one hour while learning and checkpoint handling run asynchronously.**

***Index Terms*— Robot Manipulation, Human-in-the-loop, Online Post-Training**

I. Introduction

Modern robot manipulation increasingly relies on large pretrained robot foundation models [1], [2], [3], [4], [5], [6]. A common post-training paradigm adapts these pretrained priors to a target task using offline human teleoperation data [1], [7], producing an initially deployable policy. However, real-robot failures often arise in off-demonstration states induced by the policy itself, including contact-rich interactions, recovery situations, and precision-critical configurations. Covering such rare but deployment-critical states by simply collecting additional offline teleoperation episodes is inherently data-inefficient [8], [9], [7].

Human-in-the-loop learning offers a natural mechanism for covering these policy-induced states, as human operators can intervene during failures or challenging recovery behaviors. However, existing approaches often treat deployment primarily as a provider of expert action labels for supervised learning: human-intervention segments are used as demonstrations, while policy-generated segments are discarded or treated as unlabeled data [8], [10], [11], [12]. This limits

*Equal contribution.
†Corresponding author (lizhangfd@fudan.edu.cn).

data efficiency, especially when most deployment experience is generated by the policy itself. Moreover, offline-only post-training methods [10], [11], [13], [12], [14] typically follow a collect-train-deploy cycle, preventing them from immediately incorporating deployment experience and thereby slowing adaptation under distribution shift. Online reinforcement learning leverages both human interventions and policy-generated rollouts, bringing policy updates closer to deployment [15], [16]. However, these methods often rely on value functions or critics trained from sparse episode-level success-or-failure feedback. Such weak supervision makes it difficult to stably evaluate high-dimensional action chunks, increasing both training complexity and instability. Other systems reduce this burden by updating only small policy modules or by shifting annotation away from real-robot rollouts [15], [17], [18]. While effective in constrained settings, these design choices limit the potential of online post-training to improve large robot foundation models across diverse manipulation tasks. Existing methods therefore lack a mechanism for promptly incorporating deployment-time intervention and execution signals without deferring learning to a subsequent offline training round or relying on difficult-to-estimate value functions.

Our central insight is that, once a pretrained policy is warm-started on a target task, deployment need not be a data-collection stage before the next offline retraining round. It can instead serve as a continual learning process that produces structured training signals. Human interventions and policy executions reveal what the current policy can accomplish and which state regions still require repair. Deployment data should therefore be organized as provenance-structured online experience rather than flattened into additional expert demonstrations.

Building on this insight, we introduce **TIDE** (Training In DEployment), a human-in-the-loop online post-training system for pretrained manipulation policies. As shown in Fig. 1, TIDE places the robot, the human operator, and the learner in a single deployment loop: the robot executes action chunks generated by the current policy, the operator intervenes only when necessary, and the system records the executed action at each step together with its provenance, i.e., whether it was produced by the human or the policy. Provenance records execution source only; it is not a reward, correctness label, or quality score. The learner continuously updates all trainable policy modules using both offline task data and the growing online dataset, and returns updated checkpoints to the robot at episode boundaries. Rather than treating mixed online episodes as homogeneous expert demonstrations, TIDE restructures them into pure-provenance chunks. These labels enable provenance-conditioned supervised training without a separate reward model, critic, or action scorer.

We validate TIDE on four real-robot manipulation tasks spanning precision contact, deformable-object manipulation, constrained placement, and recovery from policy-induced states. Across these tasks, TIDE improves no-takeover task performance over the warm-start policy and matched additional offline demonstrations. It performs comparably to the Fixed Policy baseline while making updates available during deployment. The observed intervention ratio also decreases over deployment, indicating lower human-supervision burden without serving as an autonomous-performance measure.

This paper makes three main contributions: (1) **TIDE**, a human-in-the-loop online post-training system that keeps real-robot execution, human takeover, learning, and policy redeployment in one session; (2) provenance-conditioned joint optimization of all trainable policy modules, turning mixed human-policy deployment data into structured source supervision; and (3) real-robot evidence across four manipulation tasks showing improved no-takeover task performance and a declining observed intervention ratio.

II. Related work

A. Robot foundation models for manipulation

Modern manipulation policies are increasingly built on large pretrained robot foundation models that map observations and language instructions to actions [19], [4], [2], [3], [1], [20], [21]. RT-1 and RT-2 [19], [4], OpenVLA [2], and Octo [3] established this recipe for broad instruction-conditioned manipulation, while recent generalist and humanoid systems extend it across embodiments and physical settings [20], [21]. Progress has also come from more capable action interfaces, including flow-matching action experts over continuous action chunks [1], [14], [22], [23], and efficient action tokenization [24], [25]. In parallel, models that couple action generation with future visual state prediction use video-generation priors to improve robustness to unfamiliar skills and embodiments [5], [6], [26], [27], [28], [29], [30], [31]. These advances rely on increasingly large cross-embodiment datasets for robot learning [32], [33], [34], [35]. Yet deployed policies remain bounded by training-distribution coverage: small errors compound into out-of-distribution states, and precision-critical subtasks can resist improvement even after many additional teleoperation episodes [9], [7]. This limitation motivates post-training procedures that refine pretrained policies through interaction with the deployment environment.

B. Human-in-the-loop post-training

Human-in-the-loop learning extends imitation learning by allowing a human to intervene on states visited by the current policy, directly addressing covariate shift and compounding errors [8], [36], [10]. Subsequent systems improve when interventions are triggered, how mixed demonstration and intervention data are weighted, what corrective data are collected, or how disruptive takeover is [11], [13], [37], [9], [38], [39], [40], [41], [42], [43], [44]. Related methods extract different learning signals from interventions: RLIF treats intervention signals as rewards for off-policy reinforcement learning, MILE models the intervention process and uses both intervention and non-intervention feedback, and Sirius reweights deployment data with approximated human trust for weighted behavioral cloning [45], [46], [13]. Many interactive imitation-learning systems still update models

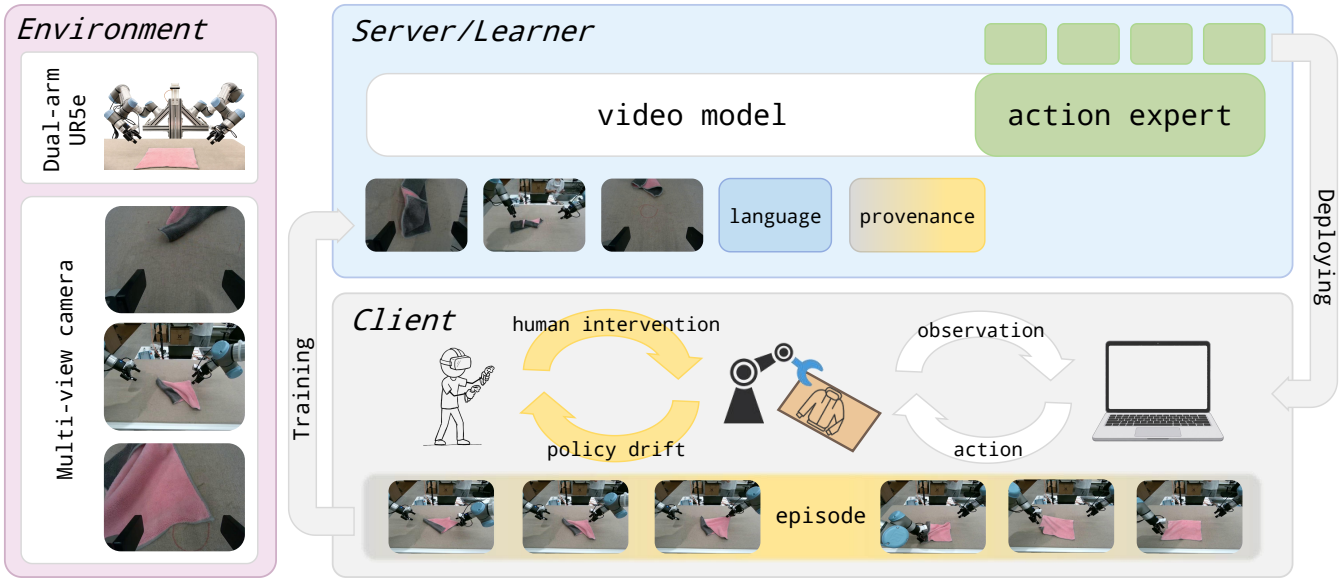


Fig. 2. **The pipeline of TIDE.** The robot executes policy actions with human takeover when needed. The system records executed actions and labels their human or policy source. The learner combines this online data with offline demonstrations to update the video model and action expert using provenance conditioning. Updated policies return to the robot at episode boundaries, while data collection and learning proceed asynchronously within the same deployment session.

in batches. RoboCopilot [12] moves toward continual fine-tuning within an interactive collection loop, but its workflow remains round-based and the deployed policy still comes from batched retraining. RECAP [14] scales full-model HIL improvement with advantage conditioning over demonstrations, rollouts, and corrections, but it still operates through phase-based offline updates and leaves a fully concurrent online framework as future work.

Recent systems move policy improvement closer to deployment, but each keeps only part of the desired closed loop. RL Token [15] performs online learning on top of a frozen pretrained policy by training a lightweight actor-critic rather than updating the full backbone. Value-based routes must estimate action quality from sparse outcome feedback [15], [47], [48], [49], which is difficult for high-dimensional action chunks in real robots. Hi-ORS [18] avoids a learned critic through outcome-based rejection sampling and supervised updates, but still relies on trajectory-level outcome supervision; ConRFT [50] instead retains a Q-function and gates its online consistency-policy fine-tuning with human interventions. RoboPocket [17] accelerates policy iteration by visualizing predicted policy trajectories in AR and collecting human corrections without executing those predictions on the physical robot.

TIDE targets a complementary setting in which real-robot deployment remains in the data loop, the video backbone, cross-modal adapter, and action expert are jointly updated, and supervision is organized by action-source provenance rather than learned values or trajectory-level reward labels.

III. Method

TIDE builds on a pretrained world-action policy consisting of a video-generation backbone Φ and a flow-matching action expert v_θ^A. Given an observation o_t and a language instruction q, the backbone produces a compact world prior $z_t = \Phi(o_t, q)$. The action expert cross-attends to z_t and the robot state s_t to generate a 50-step action chunk $A_t = (a_t, \ldots, a_{t+49})$, while the backbone predicts future video frames over the same horizon. Before deployment, we warm-start all trainable policy modules on offline human teleoperation data for the target task.

This warm-started policy provides an executable initialization, but deployment can still expose states that are not covered by offline demonstrations. TIDE turns these deployment-time states into training signals within the same session. We first describe the training-in-deployment runtime, which integrates robot execution, provenance recording, online learning, and checkpoint handoff in a single loop (Section III-A). We then explain how execution records are converted into provenance-labeled chunks for provenance-conditioned training (Section III-B). An overview of the pipeline is shown in Fig. 2.

A. Training-in-deployment system

The training-in-deployment system couples a real-robot client, an inference-and-recording server, and an online learner within a single deployment session. The client executes actions from either the policy or the human operator and streams executed-action records to the server. The server handles policy inference, stores the incoming records, and makes them available to the learner. The learner updates all trainable policy modules online and returns new policy checkpoints at episode boundaries.

a) Robot client: The robot client is responsible for low-latency real-robot control. At each control step, it collects observations, requests action chunks from the server, and ex-

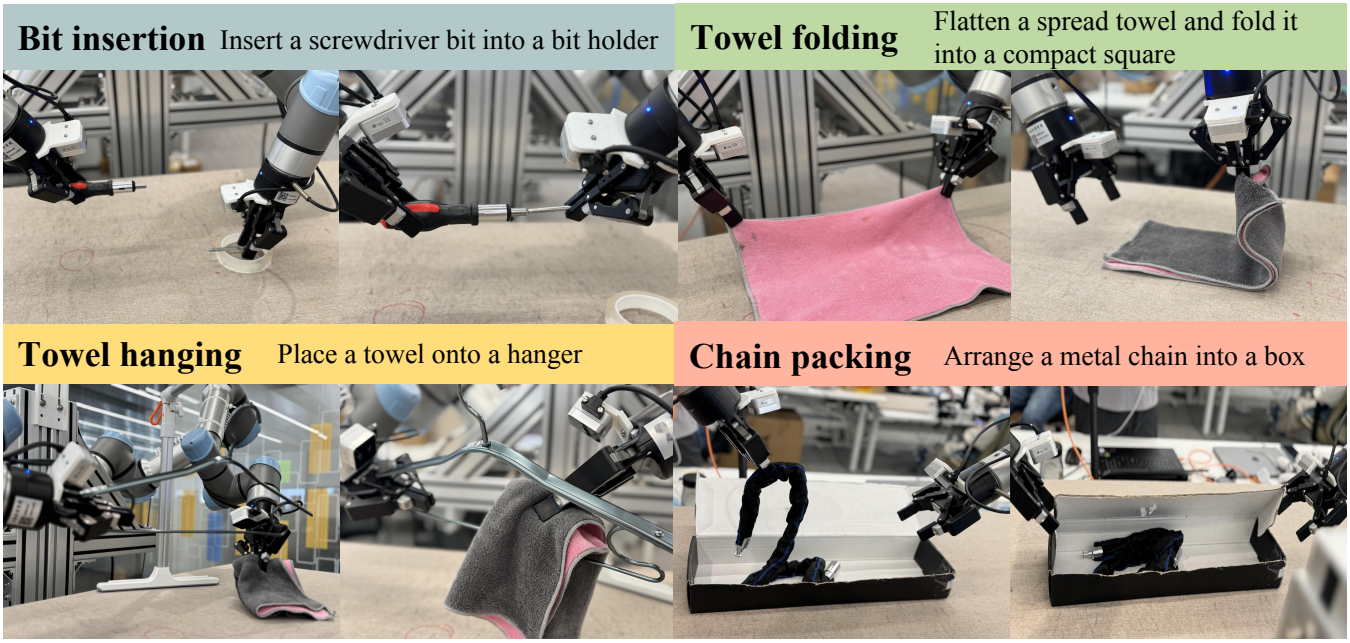


Fig. 3. **Real-world evaluation tasks.** Initial and successful final states for four bimanual manipulation tasks.

ecutes policy actions unless the operator intervenes. Inspired by RTC [51], [52], the client adopts asynchronous real-time chunking: committed actions continue to execute while future actions are refreshed in the background. This design reduces inference-induced stalls and smooths transitions between consecutive action chunks. During takeover, VR delta control allows the operator to command incremental motions around the current robot state and return control to the policy without resetting the episode. Each handoff clears queued policy chunks to prevent stale actions from being executed. The client streams executed actions and intervention flags asynchronously to the server, keeping data recording off the critical control path.

b) Inference and recording server: The inference-and-recording server mediates between real-robot control and online training. It centralizes GPU-based policy inference and writes online episodes to a shared dataset, while leaving low-latency control to the robot client. For each client query, the server runs the active checkpoint on the current observation and returns an action chunk. In parallel, it appends execution records and manages episode commit or discard. At episode end, the human operator manually confirms task completion. Before upload, the server commits only confirmed-successful episodes; failed, timed-out, aborted or stopped, reset-before-completion, re-recorded, stale, and unrecovered episodes are discarded. To ensure rollout consistency, the server adopts learner-produced checkpoints only at episode boundaries, so that each rollout is associated with a single policy generation.

c) Online learner: The online learner continuously incorporates deployment experience into the pretrained policy during the same deployment session. As committed records arrive, it reads from the growing online dataset and jointly updates the video diffusion backbone, cross-modal adapter/Q-former, action expert, and source embeddings with offline task data; the pretrained text encoder and video VAE remain frozen. The learner allocates 0.2 optimizer steps per newly committed control frame and publishes a checkpoint every five committed episodes. The server prefetches new weights and adopts them only at the next episode boundary. This mechanism returns model improvements to the robot during deployment while ensuring that each rollout is generated by a single policy generation.

B. *Provenance-conditioned learning*

a) Provenance-labeled chunks: The execution loop described above produces a continuous stream of mixed deployment episodes containing both human interventions and policy executions. To convert these episodes into supervision, TIDE assigns provenance labels at the action-chunk level rather than treating an entire episode as a single demonstration. Human-executed chunks provide supervision for failure recovery and difficult states, while policy-executed chunks provide additional learning signal from behaviors that the current policy can already perform. Chunk-level provenance labels allow TIDE to use both sources of deployment data without flattening mixed episodes into homogeneous expert demonstrations.

The provenance labels are derived directly from signals recorded during execution. At each control step, TIDE stores the action sent to the robot and whether that action was produced by the human operator or by the policy. Provenance encodes action-source identity only, not action quality. Episode success is used only to decide whether an episode enters the online training dataset; it is not a model condition, reward, provenance label, or training target. Given an action

chunk A, TIDE assigns

$$\ell(A) = \begin{cases} 0, & A \text{ is fully human-executed,} \\ 1, & A \text{ is fully policy-executed,} \\ \varnothing, & A \text{ mixes human and policy steps.} \end{cases} \tag{1}$$

A 50-step target-action horizon receives a human or policy label only when all 50 actions share that source. Horizons labeled $\varnothing$ contain both sources and are excluded from provenance-conditioned training, with no additional exclusion window around a handoff.

b) Provenance-conditioned online training: Given the provenance-labeled chunks defined above, TIDE trains all trainable policy modules with a provenance-conditioned flow-matching objective. We map each provenance label ℓ to a learnable embedding c_ℓ and condition both the video-generation backbone and the action expert on this embedding. The backbone takes (o_t, q, c_ℓ) as input to produce a provenance-conditioned world prior $z_\ell = \Phi(o_t, q; c_\ell)$ and predict the future-video velocity v_ϕ^I. The action expert then conditions on (z_ℓ, s_t, c_ℓ) to predict the action velocity v_θ^A. In both networks, c_ℓ is added to the flow-time embedding. All reported online autonomous rollouts and final no-takeover evaluations fix the source condition to the human mode, $\ell = 0$.

We use a linear-Gaussian probability path. Given a flow time $\tau \in [0, 1]$ and independent noises $\varepsilon_I, \varepsilon_A \sim \mathcal{N}(0, \mathbf{I})$, we construct noisy interpolants $I_t^\tau = \tau I_t + (1-\tau)\varepsilon_I$ and $A_t^\tau = \tau A_t + (1-\tau)\varepsilon_A$, with target velocities $I_t - \varepsilon_I$ and $A_t - \varepsilon_A$, respectively. The training objective is

$$\mathcal{L}_{\text{WAM}} = \mathbb{E}\Big[\big\|v_\phi^I(I_t^\tau, \tau \mid o_t, q, c_\ell) - (I_t - \varepsilon_I)\big\|_2^2 + \big\|v_\theta^A(A_t^\tau, \tau \mid z_\ell, s_t, c_\ell) - (A_t - \varepsilon_A)\big\|_2^2\Big]. \tag{2}$$

The training data combine offline teleoperation data, treated as human-provenance chunks with $\ell = 0$, and the pure-provenance online chunks described above. The learner maintains separate offline and online buffers. Online samples receive a 5:1 weight relative to offline samples, corresponding to selection probabilities $5/6$ and $1/6$ between the two buffers. Preliminary tuning selected $\alpha = 5$, which is fixed across all reported tasks and variants. Within the online buffer, human-executed and policy-executed chunks are balanced so that sparse intervention data are not overwhelmed by autonomous execution.

We also apply two task-preserving data augmentations while jointly optimizing all trainable policy modules. Space mirroring reflects observations, robot states, and actions across the workspace symmetry plane when the task remains valid under this transformation, increasing coverage of left and right spatial configurations. Time scaling resamples action chunks and corresponding future observations at different execution rates, exposing the model to moderate speed variation. Each augmented sample inherits the provenance label of its original chunk.

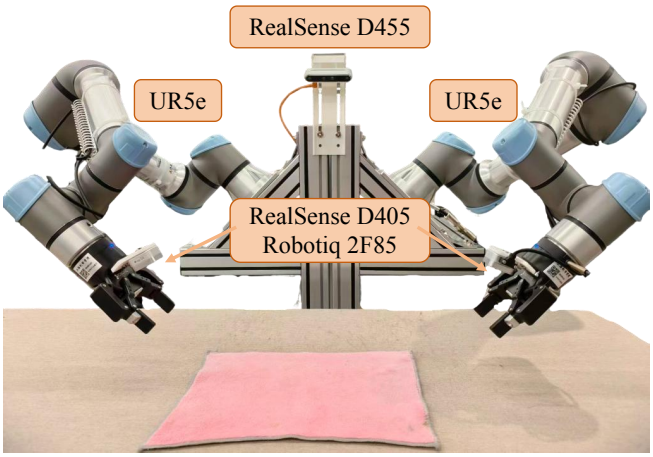


Fig. 4. **Real-robot setup.** The bimanual UR5e platform uses Robotiq 2F85 grippers, one third-person RealSense D455 camera, and two wrist-mounted RealSense D405 cameras.

IV. Experiments

A. Experimental setup

a) Robot platform: We conduct all real-world experiments on the bimanual UR5e platform shown in Fig. 4. The perception setup includes one third-person D455 camera and two wrist-mounted D405 cameras, providing both scene-level context and local contact observations near the grippers.

b) Tasks: We evaluate TIDE on four real-world bimanual manipulation tasks shown in Fig. 3. **Bit Insertion** requires inserting a screwdriver bit into a bit holder and tests precise pose control under local contact. **Towel Folding** requires flattening a spread towel and folding it into a compact square, stressing long-horizon deformable-object manipulation. **Towel Hanging** requires placing a towel onto a hanger, combining deformable manipulation with a constrained target. **Chain Packing** requires arranging a metal chain into a box, creating recovery states from local deformation and possible tangling.

c) Protocol and metrics: Every method–task pair is evaluated using 20 autonomous rollouts from the same predefined initial-state set, with no human takeover. TIDE uses the human source condition $\ell = 0$ during these evaluations. Let $c_{r,t}$ denote the number of milestones completed in rollout r for task t, and let C_t denote the total number of milestones for that task. We compute the normalized rollout score and aggregate task progress as

$$a_{r,t} = \frac{c_{r,t}}{C_t}, \qquad \bar{a}_t = \frac{1}{N}\sum_{r=1}^{N} a_{r,t}, \quad N = 20.$$

If a rollout ends because of a timeout, safety stop, or task failure, only milestones already stably completed are counted. We report binary full-task completion separately as $k/20$. A rollout is complete only if $c_{r,t} = C_t$. During HIL collection, only episodes manually confirmed as successful are committed to the online training dataset; success affects dataset inclusion only. We define human intervention ratio as the fraction of executed control steps under human takeover

and use it only as a within-task measure of supervision burden.

The task milestones are pick/insert for Bit Insertion; flatten/first fold/second fold for Towel Folding; pick hanger/place towel/hang hanger for Towel Hanging; and place chain/close lid/tuck flaps for Chain Packing.

d) Baselines: Every method starts from the same task-specific warm-start checkpoint. The offline-demo baseline uses newly collected teleoperation demonstrations matched to the amount of online data, then trains for the same number of optimizer steps as TIDE. The fixed-policy collection baseline (Fixed Policy) holds the warm-start policy fixed during HIL collection matched to TIDE by duration, then trains after collection using the same objective, buffer configuration, filtering rules, and total optimizer-update budget. TIDE instead trains and redeploys checkpoints during collection.

e) Implementation details: The reported world-action policy has 3B parameters and is trained on eight H100 GPUs. Each online optimizer step takes approximately 0.4 s on this setup. Online training jointly updates the video diffusion backbone, cross-modal adapter/Q-former, action expert, and source embeddings, while the text encoder and video VAE remain frozen. The policy predicts a 50-step, 14-dimensional bimanual joint-relative action chunk. Each predicted offset is added to the current joint state before the controller receives the resulting absolute joint target. Online samples have a 5:1 weight relative to offline samples ($\alpha = 5$), equivalent to an online-buffer selection probability of $5/6$; this preliminarily tuned setting is fixed across all reported tasks and variants. The learner takes 0.2 optimizer steps per newly committed control frame and publishes a checkpoint every five committed episodes; the server prefetches and adopts it at the next episode boundary. For each task, the continuous wall-clock interval from the start of online HIL rollouts until the final checkpoint is deployable is less than one hour. This interval includes asynchronous data processing, training, checkpoint handling, robot execution, human takeover, and episode resets, but excludes initial setup and final autonomous evaluation. Online training runs concurrently with data collection and episode resets, and its wall-clock cost is included in this reported interval. For Bit Insertion, collection comprised 41 committed episodes and 38,839 control frames (21.6 minutes at 30 Hz): 1,432 human-controlled frames (3.69%) and 37,407 policy-controlled frames.

B. Real-world experiments

a) Q1: Which action representation should be used: Action representation affects how the policy expresses contact-rich motions, so we first evaluate this choice before online post-training. We train offline-only policies with the same data, initialization, and budget, comparing eight spaces: absolute, delta, and relative actions in joint and TCP space, plus delta and relative actions in tool space. As shown in Fig. 5, joint-relative actions perform best in this controlled study and are used in all subsequent experiments.

b) Q2: Does TIDE improve autonomous task performance: Because human takeover can complete training

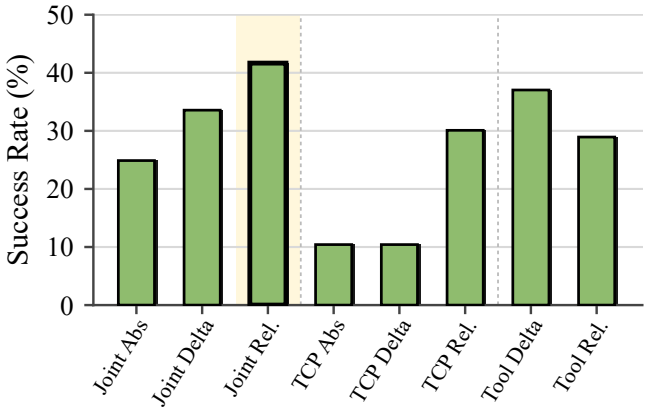


Fig. 5. **Action representation study.** Offline-only comparison of eight action spaces using the same data, initialization, and training budget. Joint-relative actions perform best and are used in subsequent experiments.

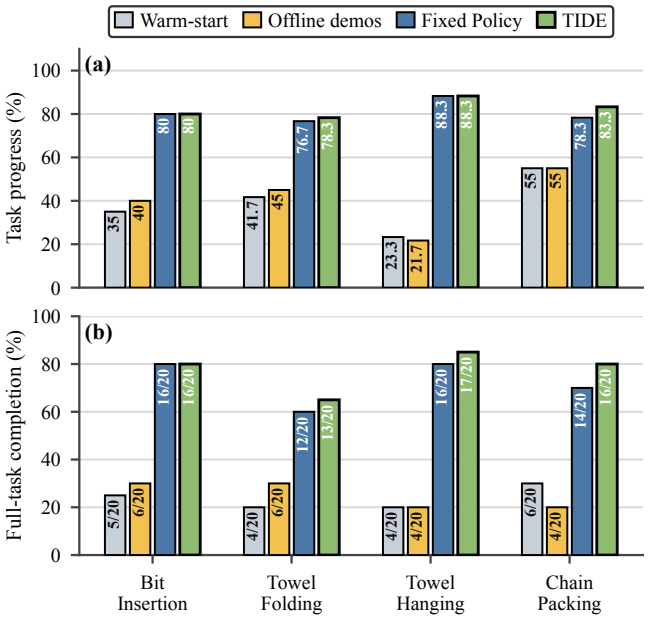


Fig. 6. **Autonomous evaluation over 20 no-takeover rollouts per method–task pair. (a)** Mean milestone-normalized task progress. **(b)** Binary full-task completion. Labels report completed rollouts out of 20.

episodes that would otherwise fail, we measure task performance only with the no-takeover evaluation described above. Figure 6 reports milestone-normalized task progress and binary full-task completion. The completion labels give the number of completed rollouts out of 20. The two metrics capture different outcomes. Progress credits intermediate milestones. Completion requires the entire task. Both Fixed Policy and TIDE numerically outperform Warm and Offline on milestone-normalized progress for all four tasks. TIDE matches Fixed Policy on Bit Insertion and Towel Hanging progress, and is higher on Towel Folding and Chain Packing. Binary completion is tied on Bit Insertion and is higher for TIDE by one rollout on both towel tasks and by two rollouts on Chain Packing. Thus, Fixed Policy and TIDE have comparable final performance under this evaluation. TIDE's

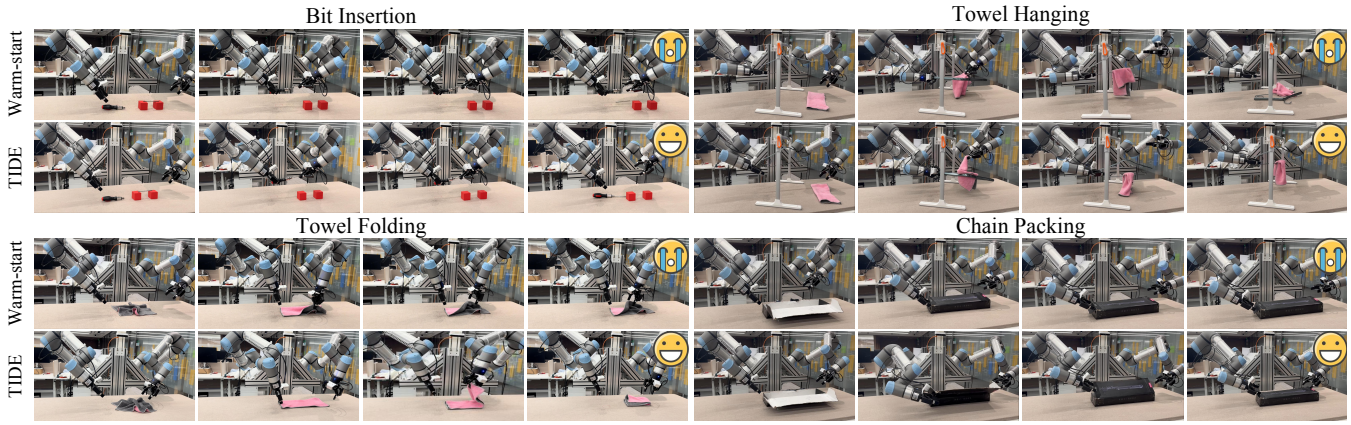


Fig. 7. **Qualitative autonomous rollouts before and after online post-training.** For each task, the top and bottom rows show no-takeover rollouts from the warm-start policy and final TIDE checkpoint, respectively, with time progressing from left to right. The warm-start policy fails to complete the tasks in these examples, whereas TIDE completes the full manipulation sequences, illustrating improved autonomous execution after online post-training.

distinction is that updated checkpoints become available during the deployment session rather than only after a separate collection, training, and redeployment cycle. Figure 7 provides illustrative before-and-after rollouts; the quantitative comparison is based on the 20-rollout evaluations above.

c) Q3: How does observed supervision burden evolve during deployment: For each HIL training episode, we compute the intervention ratio as the fraction of executed control steps under human takeover. In Fig. 8(a), gray markers show the per-episode ratio for Bit Insertion, and the red curve shows the frame-weighted ratio over the most recent five committed episodes. The observed windowed ratio decreases after the early deployment phase, indicating lower supervision burden over the session. This trace is not an autonomous-performance measure and does not by itself isolate learning from temporal factors.

d) Q4: How important is provenance-conditioned learning: Figure 8(b) isolates the role of provenance conditioning on Bit Insertion. All variants start from the same warm-start checkpoint, use the same online interaction budget, and are evaluated with 20 no-takeover rollouts from the same initial states. TIDE obtains 80.0% milestone-normalized task progress, compared with 65.0% without provenance conditioning and 67.5% when only human-executed online chunks are used. This single-task ablation supports both explicit source conditioning and the use of policy-executed chunks for Bit Insertion.

V. Conclusion

We presented TIDE, a human-in-the-loop online post-training system that turns real-robot deployment into a learning process for pretrained manipulation policies. TIDE places robot execution, human takeover, online data formation, joint optimization of all trainable policy modules, and checkpoint handoff in one deployment session. Its provenance-labeled chunks distinguish human-intervention from policy-executed behavior by source, enabling supervised online training without learned critics, trajectory-level reward labels, or action scorers. Across four real-world tasks, TIDE achieves higher

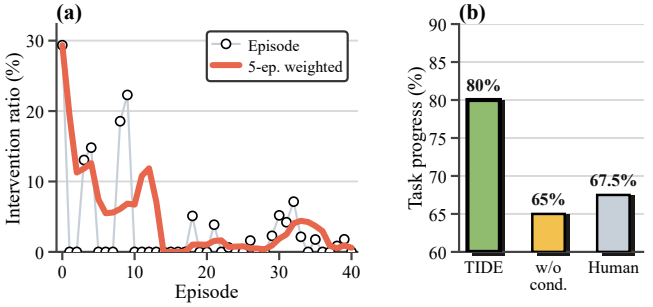


Fig. 8. **Deployment diagnostics on Bit Insertion. (a)** The intervention ratio decreases over deployment, indicating reduced human-supervision burden. **(b)** TIDE achieves higher normalized task progress than variants without provenance conditioning or using only human-executed online data, highlighting the value of source conditioning and learning from policy-executed experience.

milestone-normalized task progress and binary completion than the warm-start and additional-offline-data baselines. Its final performance is comparable to the Fixed Policy baseline, while updated checkpoints become available during deployment. The observed intervention ratio also declines over the Bit Insertion session, indicating lower supervision burden rather than serving as a separate measure of autonomy.

VI. Limitations

TIDE requires a human supervisor, and retaining only manually confirmed successful episodes may introduce selection bias. Our evaluation covers one bimanual platform, four tasks, and one world-action backbone, and does not test long-term retention or sensitivity to the online-training schedule. Generalization across platforms and architectures therefore remains unverified. We also do not measure variation across repeated online post-training sessions.

References

[1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[3] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[4] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.

[5] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, *et al.*, "World action models are zero-shot policies," *arXiv preprint arXiv:2602.15922*, 2026.

[6] Y. Shen, F. Wei, Z. Du, Y. Liang, Y. Lu, J. Yang, N. Zheng, and B. Guo, "VideoVLA: Video generators can be generalizable robot manipulators," *Advances in neural information processing systems*, vol. 38, pp. 105 974–105 998, 2025.

[7] T. Z. Zhao, J. Tompson, D. Driess, P. Florence, K. Ghasemipour, C. Finn, and A. Wahid, "ALOHA Unleashed: A simple recipe for robot dexterity," *arXiv preprint arXiv:2410.13126*, 2024.

[8] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.

[9] Z. Hu, R. Wu, N. Enock, J. Li, R. Kadakia, Z. Erickson, and A. Kumar, "RAC: Robot learning for long-horizon tasks by scaling recovery and correction," *arXiv preprint arXiv:2509.07953*, 2025.

[10] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "HG-DAgger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8077–8083.

[11] A. Mandlekar, D. Xu, R. Martín-Martín, Y. Zhu, L. Fei-Fei, and S. Savarese, "Human-in-the-loop imitation learning using remote teleoperation," *arXiv preprint arXiv:2012.06733*, 2020.

[12] P. Wu, Y. Shentu, Q. Liao, D. Jin, M. Guo, K. Sreenath, X. Lin, and P. Abbeel, "RoboCopilot: Human-in-the-loop interactive imitation learning for robot manipulation," *arXiv preprint arXiv:2503.07771*, 2025.

[13] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, "Robot learning on the job: Human-in-the-loop autonomy and learning during deployment," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1727–1742, 2025.

[14] Physical Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, *et al.*, "$\pi^*_{0.6}$: a VLA that learns from experience," *arXiv preprint arXiv:2511.14759*, 2025.

[15] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke, "RL Token: Bootstrapping online RL with vision-language-action models," *arXiv preprint arXiv:2604.23073*, 2026.

[16] J. Luo, C. Xu, J. Wu, and S. Levine, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.

[17] J. Fang, W. Chen, H. Xue, F. Zhou, T. Le, Y. Wang, Y. Zhang, J. Lv, C. Wen, and C. Lu, "Robopocket: Improve robot policies instantly with your phone," *arXiv preprint arXiv:2603.05504*, 2026.

[18] G. Lu, R. Zhao, H. Lin, H. Zhang, and Y. Tang, "Human-in-the-loop online rejection sampling for robotic manipulation," *arXiv preprint arXiv:2510.26406*, 2025.

[19] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, *et al.*, "RT-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.

[20] Gemini Robotics Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, T. Armstrong, A. Balakrishna, R. Baruch, M. Bauza, M. Blokzijl, *et al.*, "Gemini Robotics: Bringing AI into the physical world," *arXiv preprint arXiv:2503.20020*, 2025.

[21] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, *et al.*, "GR00T N1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

[22] T. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems*, 2023.

[23] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[24] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, "RDT-1B: a diffusion foundation model for bimanual manipulation," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 29 982–30 009.

[25] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, "FAST: Efficient action tokenization for vision-language-action models," *arXiv preprint arXiv:2501.09747*, 2025.

[26] J. Guo, Q. Li, P. Li, Z. Chen, N. Sun, Y. Su, H. Wang, Y. Zhang, X. Li, and H. Liu, "Unified 4d world action modeling from video priors with asynchronous denoising," *arXiv preprint arXiv:2604.26694*, 2026.

[27] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava, "Mimic-Video: Video-action models for generalizable robot control beyond VLAs," *arXiv preprint arXiv:2512.15692*, 2025.

[28] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel, "Learning universal policies via text-guided video generation," *Advances in neural information processing systems*, vol. 36, pp. 9156–9172, 2023.

[29] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong, "Unleashing large-scale video generative pre-training for visual robot manipulation," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 10 641–10 662.

[30] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen, "Video prediction policy: A generalist robot policy with predictive visual representations," *arXiv preprint arXiv:2412.14803*, 2024.

[31] Y. Tian, S. Yang, J. Zeng, P. Wang, D. Lin, H. Dong, and J. Pang, "Predictive inverse dynamics models are scalable learners for robotic manipulation," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 92 033–92 052.

[32] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, *et al.*, "DROID: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.

[33] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch, A. W. He, V. Myers, M. J. Kim, M. Du, *et al.*, "BridgeData V2: A dataset for robot learning at scale," in *Conference on Robot Learning*. PMLR, 2023, pp. 1723–1736.

[34] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, *et al.*, "Open X-Embodiment: Robotic learning datasets and RT-X models: Open X-Embodiment collaboration 0," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6892–6903.

[35] Q. Bu, J. Cai, L. Chen, X. Cui, Y. Ding, S. Feng, S. Gao, X. He, X. Hu, X. Huang, *et al.*, "AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems," *arXiv preprint arXiv:2503.06669*, 2025.

[36] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "DART: Noise injection for robust imitation learning," in *Conference on robot learning*. PMLR, 2017, pp. 143–156.

[37] H. Liu, Y. Zhang, V. Betala, E. Zhang, J. Liu, C. Ding, and Y. Zhu, "Multi-task interactive robot fleet learning with visual world models," *arXiv preprint arXiv:2410.22689*, 2024.

[38] W. Wang, J. Song, C. Liu, J. Ma, S. Feng, J. Wang, Y. Jiang, K. Chen, S. Zhan, Y. Wang, *et al.*, "Genie Centurion: Accelerating scalable real-world robot training with human rewind-and-refine guidance," *arXiv preprint arXiv:2505.18793*, 2025.

[39] R. Hoque, A. Mandlekar, C. Garrett, K. Goldberg, and D. Fox, "IntervenGen: Interventional data generation for robust and data-efficient robot imitation learning," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 2840–2846.

[40] X. Xu, Y. Hou, Z. Liu, and S. Song, "Compliant residual DAgger: Improving real-world contact-rich manipulation with human corrections," *Advances in Neural Information Processing Systems*, vol. 38, pp. 154 774–154 796, 2025.

[41] J. Zhang and K. Cho, "Query-efficient imitation learning for end-to-end autonomous driving," *arXiv preprint arXiv:1605.06450*, 2016.

[42] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg, "ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning," *arXiv preprint arXiv:2109.08273*, 2021.

[43] R. Hoque, A. Balakrishna, C. Putterman, M. Luo, D. S. Brown, D. Seita, B. Thananjeyan, E. Novoseller, and K. Goldberg, "LazyDAgger: Reducing context switching in interactive imitation learning," in *2021 IEEE 17th international conference on automation science and engineering (case)*. IEEE, 2021, pp. 502–509.

[44] K. Menda, K. Driggs-Campbell, and M. J. Kochenderfer, "EnsembleDAgger: A bayesian approach to safe imitation learning," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 5041–5048.

[45] J. Luo, P. Dong, Y. Zhai, Y. Ma, and S. Levine, "RLIF: Interactive imitation learning as reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2311.12996

[46] Y. Korkmaz and E. Bıyık, "MILE: Model-based intervention learning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 15 673–15 679. [Online]. Available: https://doi.org/10.1109/ICRA55743.2025.11128018

[47] Y. Wang, X. Li, P. Xie, P. Yang, B. Nie, Y. Cai, Q. Zhang, C. Qu, J. Wu, J. Song, *et al.*, "Learning while deploying: Fleet-scale reinforcement learning for generalist robot policies," *arXiv preprint arXiv:2605.00416*, 2026.

[48] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, "SERL: A software suite for sample-efficient robotic reinforcement learning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 16 961–16 969.

[49] Y. Chebotar, Q. Vuong, K. Hausman, F. Xia, Y. Lu, A. Irpan, A. Kumar, T. Yu, A. Herzog, K. Pertsch, *et al.*, "Q-Transformer: Scalable offline reinforcement learning via autoregressive Q-functions," in *Conference on Robot Learning*. PMLR, 2023, pp. 3909–3928.

[50] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, "ConRFT: A reinforced fine-tuning method for VLA models via consistency policy," *arXiv preprint arXiv:2502.05450*, 2025.

[51] K. Black, M. Y. Galliker, and S. Levine, "Real-time execution of action chunking flow policies," in *Advances in Neural Information Processing Systems*, 2025, arXiv:2506.07339.

[52] K. Black, A. Z. Ren, M. Equi, and S. Levine, "Training-time action conditioning for efficient real-time chunking," *arXiv preprint arXiv:2512.05964*, 2025.